NATIONAL ECONOMICS UNIVERSITY
FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

TECHNICAL REPORT:
EVENT MANAGEMENT SYSTEM (EMS)

Subject: Introduction to Databases
Class: DS66B - FDA
Instructor: Trần Hùng
Student: Nguyễn Bảo Khánh
Project Source (GitHub):
https://github.com/baokhanh1410/event-management-system

May 5th, 2026, Hanoi

TABLE OF CONTENT

1. Abstract / Introduction

The Event Management System (EMS) is a comprehensive platform developed to streamline the organization of events, track financial metrics, and leverage Data Science methodologies to significantly enhance the overall user experience. The primary goal of this project is to create an integrated ecosystem that bridges robust database management with intuitive frontend interfaces and intelligent machine learning capabilities.

Core features of the system include:

- **Event Lifecycle Management:** The system allows authorized users to seamlessly create, edit, and coordinate various events.
- **Registration & Attendance:** The platform supports guest registration and provides real-time tracking of guest check-ins.
- **Financial Tracking:** Users can continuously monitor projected budgets against actual incurred costs.
- **Role-Based Access Control (RBAC):** The system enforces security and operational hierarchy by assigning distinct permissions to Admin, Staff, Organizer, and Guest roles.
- **Machine Learning Integration:** An XGBoost-based recommendation engine is deployed to suggest highly relevant events to users based on historical data.
- **Database Optimization:** The underlying MySQL architecture utilizes indexes, views, triggers, and stored procedures to guarantee high performance and strict data integrity.

2. Technology Stack

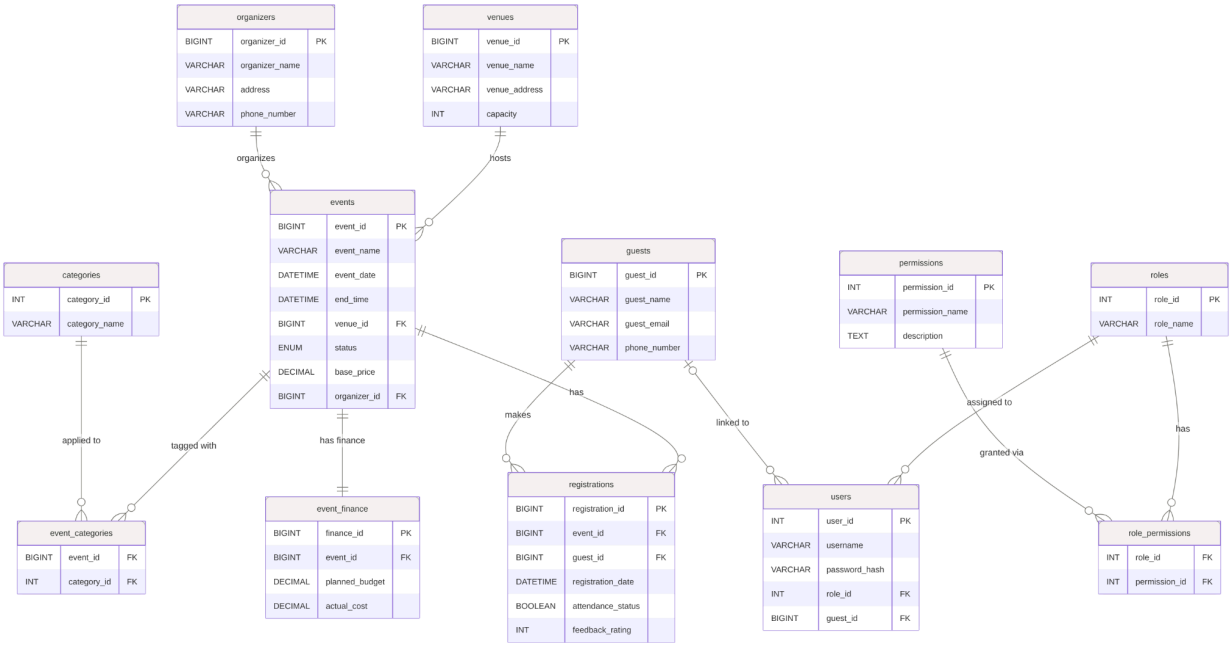
The project relies on a modern, Python-centric technology stack to manage operations from the user interface down to the machine learning algorithms:

- **Frontend/UI:** Streamlit is utilized to create an interactive, responsive web interface for managing events and viewing analytics.
- **Backend/Logic:** Python serves as the core language, handling business logic, authentication via bcrypt, and data manipulation.
- **Database:** MySQL acts as the relational storage layer, utilizing stored procedures and views to ensure high performance and data integrity.
- **Data Science:** XGBoost and Scikit-Learn power the recommendation engine, processing historical data to enhance guest engagement.
- **Infrastructure & Utilities:** The system integrates SQLAlchemy for ORM-based database management and the Faker library to generate realistic datasets for testing and validation.

3. Database Design & Optimization

The foundation of the EMS is a rigorously structured MySQL database defined within the `sql` directory.

- **Schema Definition (`schema.sql`):** The schema explicitly maps out tables including `events`, `categories`, `users`, `roles`, `permissions`, `registrations`, `event_finance`, and `venues`. Crucially, it establishes necessary indexing for query optimization and foreign key constraints to strictly maintain relational data integrity.
- **Stored Procedures:** Advanced database logic is localized on the database server to ensure consistency. The `sp_check_in_guest` procedure securely handles the logic of marking a guest as "Checked-in" while simultaneously preventing duplicate check-ins or the checking-in of non-existent registrations.
- **Analytical Views:** To optimize complex queries for the UI dashboards, custom views are deployed:
    - `vw_event_attendance_summary`: This view aggregates registration and attendance records to instantly provide total registrations and attendance rates for any given event.
    - `vw_finance_summary`: This view calculates critical financial metrics by aggregating actual costs and revenues (driven by check-ins/registrations) to dynamically compute the profit and variance per event.



4. System Architecture & Backend Logic

The backend logic is entirely encapsulated within the `src` directory, ensuring modularity and clear separation of concerns.

- **Authentication and Security (`auth.py`):** This module handles user authentication and authorization. Passwords are securely hashed and verified utilizing the `bcrypt` library. The module also manages the registration flow for new Guest users and implements the `has_permission()` function to enforce Role-Based Access Control (RBAC) across the application.
- **Database Connections (`connection.py`):** Database connectivity is established using SQLAlchemy. This file reads database credentials from environment variables (`.env`) to generate a singleton SQLAlchemy engine and manages session objects (`Session()`) for executing transactions.
- **Data Operations (`crud.py`):** All Create, Read, Update, and Delete (CRUD) operations are managed here, acting as the bridge between the Python application and the MySQL database. This module relies heavily on SQLAlchemy raw text queries and Pandas for tasks spanning event management, attendance tracking, financial summaries, and gathering historical data for the recommendation model.
- **Data Generation (`sample_data.py`):** To thoroughly test the system, the `Faker` library is utilized to procedurally generate realistic dummy data, such as venues, organizers, guests, and budgets. This script writes the `INSERT` SQL statements directly into `sql/seed.sql` and handles the logic for realistic pricing and attendance rates.

5. Machine Learning Integration

The EMS integrates predictive analytics, demonstrating the powerful intersection of Data Science and operational databases.

- **Model Architecture (`ml_models.py`):** The system employs an `XGBClassifier` (XGBoost) model that acts as a sophisticated recommendation engine.
- **Feature Engineering:** The module utilizes One-Hot Encoding during the preprocessing phase to translate categorical event features into a format suitable for the algorithm.
- **Predictive Capabilities:** The model uses historical data and the `get_recommended_events()` function to predict attendance likelihood, suggesting top-N relevant events tailored to each user.

6. User Interface & Application Flow

The frontend is constructed using Streamlit, with modular pages located in the `pages` directory. The UI dynamically adapts based on the active user's role and permissions.

- **Authentication Portal (`0_Login.py`):** Serving as the application's entry point, this page manages user authentication and state. It supports login functionality for Admin, Staff, and Guests, and provides a seamless signup tab for new users.
- **Events Management (`1_Events.py`):** This multifaceted page acts as the primary hub for event discovery and organization.
    - **Guests:** Can view a comprehensive public dataframe of events, track their successfully registered events under "My Events", and receive personalized XGBoost Machine Learning recommendations.
    - **Admin/Staff:** Gain access to specialized forms that allow for the creation of new events or the modification of existing event parameters.
- **Attendance Tracking (`2_Attendence.py`):** Dedicated to logistical operations, this page presents staff with registration lists and check-in tools. It enables authorized users to manually register guests and mark them as arrived, while also providing high-level attendance metrics and check-in rates for administrative review.
- **Financial Dashboard (`3_Finance.py`):** Access to this page is strictly limited to users possessing `manage_finance` or `view_analytics` permissions. It offers a sophisticated overview through KPI cards displaying Total Budget, Cost, Revenue, and Net Profit. The interface utilizes bar charts and conditionally formatted data tables to clearly highlight profit (green) and loss (red), while also permitting authorized users to directly edit planned budgets and actual costs.

7. Conclusion

The Event Management System effectively fulfills the requirements of a modern, data-driven application by harmonizing relational database management, machine learning, and interactive user interface design. By leveraging MySQL's structural integrity (Views, Procedures) alongside Python's flexibility (SQLAlchemy, XGBoost) and Streamlit's rapid deployment capabilities, the EMS delivers a comprehensive solution that serves organizers, staff, and guests efficiently. The inclusion of a customized recommendation engine elevates the platform beyond traditional CRUD applications, showcasing a sophisticated, academic approach to leveraging stored data for predictive user experiences.